

Data structures & algorithms workshops

Introduction to Codility exam

- Prepared by: Magdalena Zawada
- Date: 25/09/20X26



AGENDA

What I'll Cover



Algorithms & Big-O Notation

What they are, and why they matter



Comparing Growth Rates

$O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$ side by side



Core Data Structures

list, dict, str, set, tuple



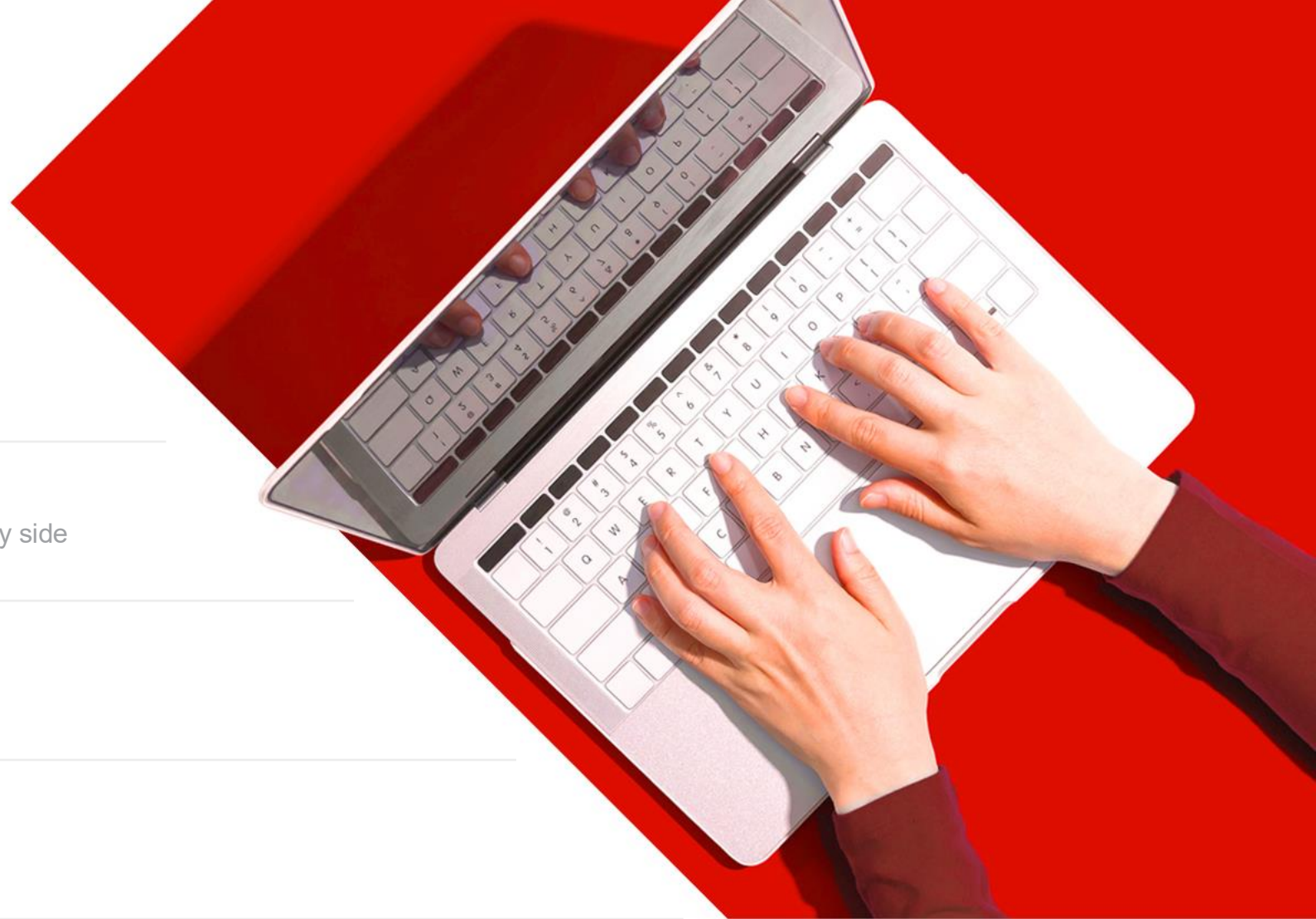
Search, Add, Delete

The basic operations - and what they cost



Cheat-Sheet Summary

Everything on a single slide



What Is an Algorithm? What Is Big-O?



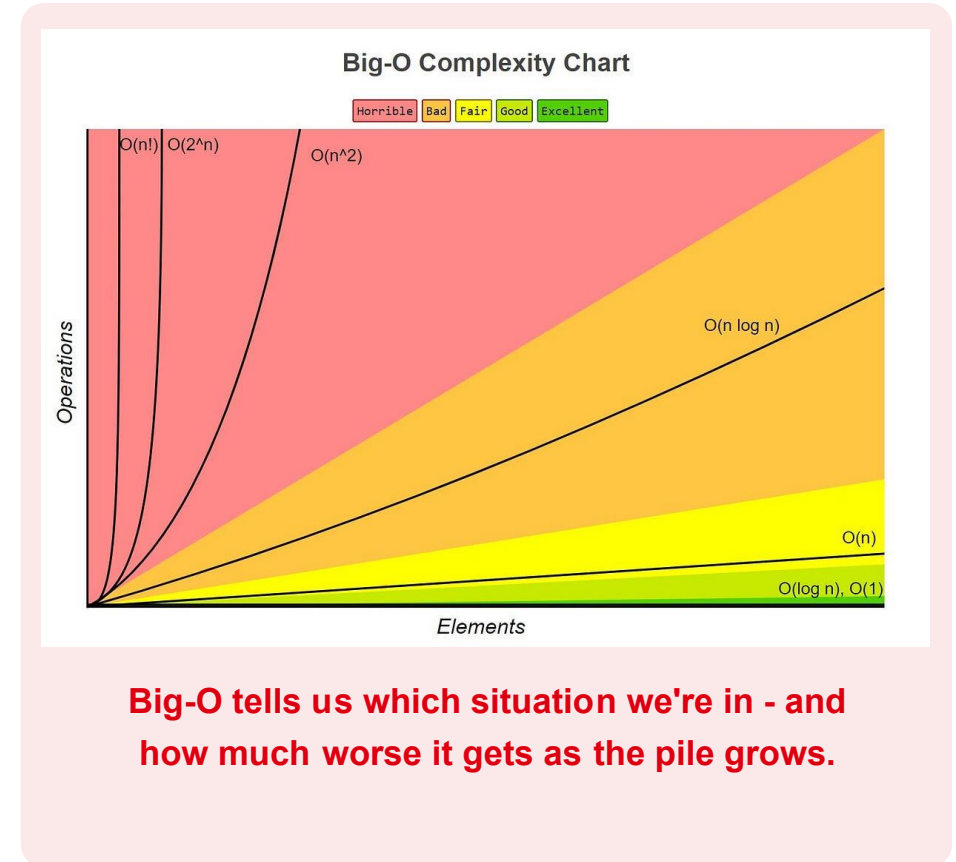
Algorithm

A step-by-step recipe for solving a problem. Sorting a list, finding a name, adding up a bill - these are all algorithms.

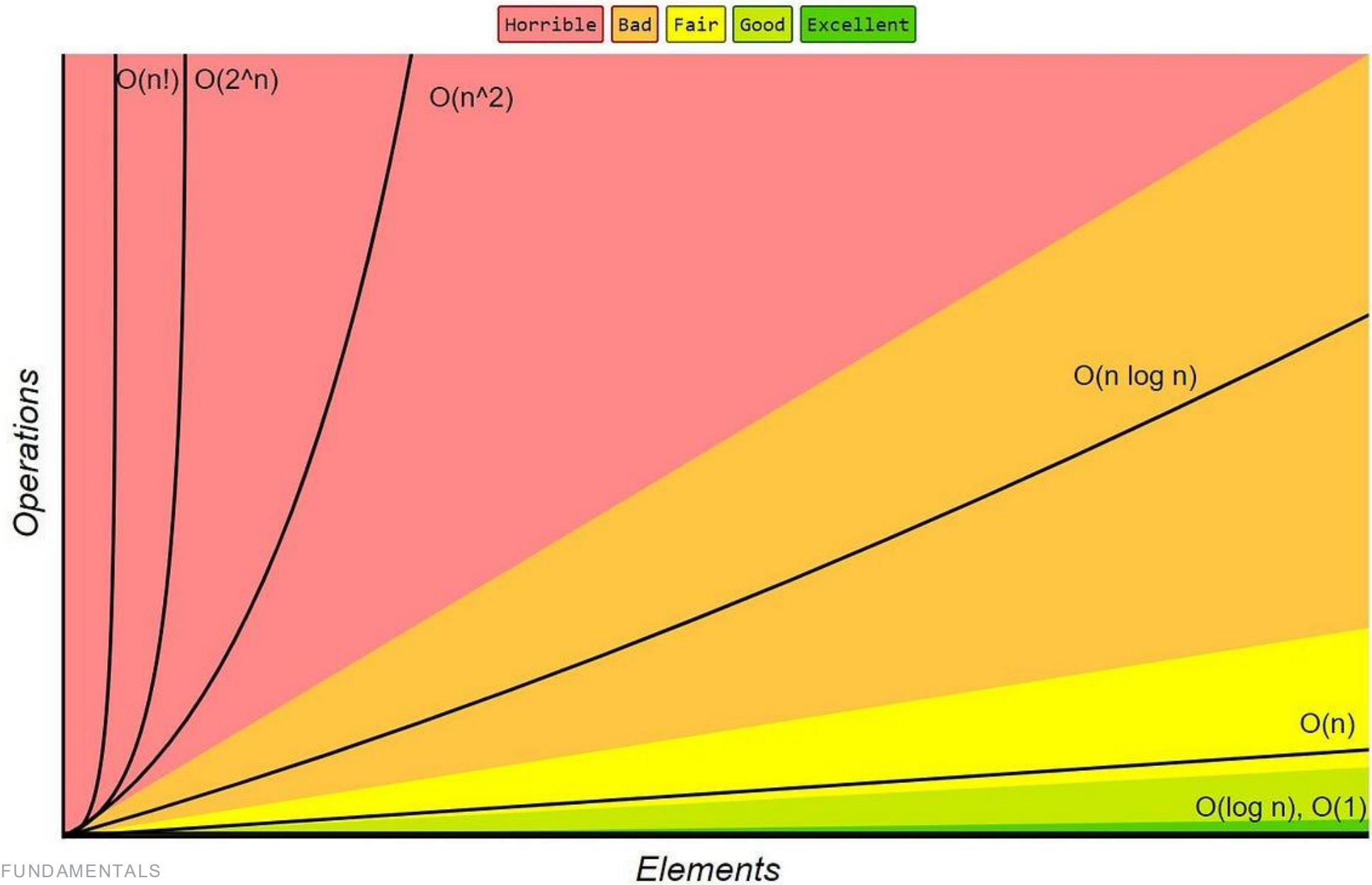
Big-O Notation

A simple way to describe how the running time (or memory use) of an algorithm grows as the input gets bigger. It isn't about exact seconds - it's about the SHAPE of growth.

That's why we write $O(n)$ instead of "3 milliseconds" - the same code runs at different speeds on different computers, but its growth shape never changes.



Big-O Complexity Chart



Common Growth Rates, In Plain English

$O(1)$

Constant

Grabbing the top card from a deck - the deck's size doesn't matter.

$O(\log n)$

Logarithmic

Finding a name in a phone book by jumping to the middle each time.

$O(n)$

Linear

Reading every page of a book to find one sentence.

$O(n \log n)$

Linearithmic

Efficiently sorting a shuffled deck of cards.

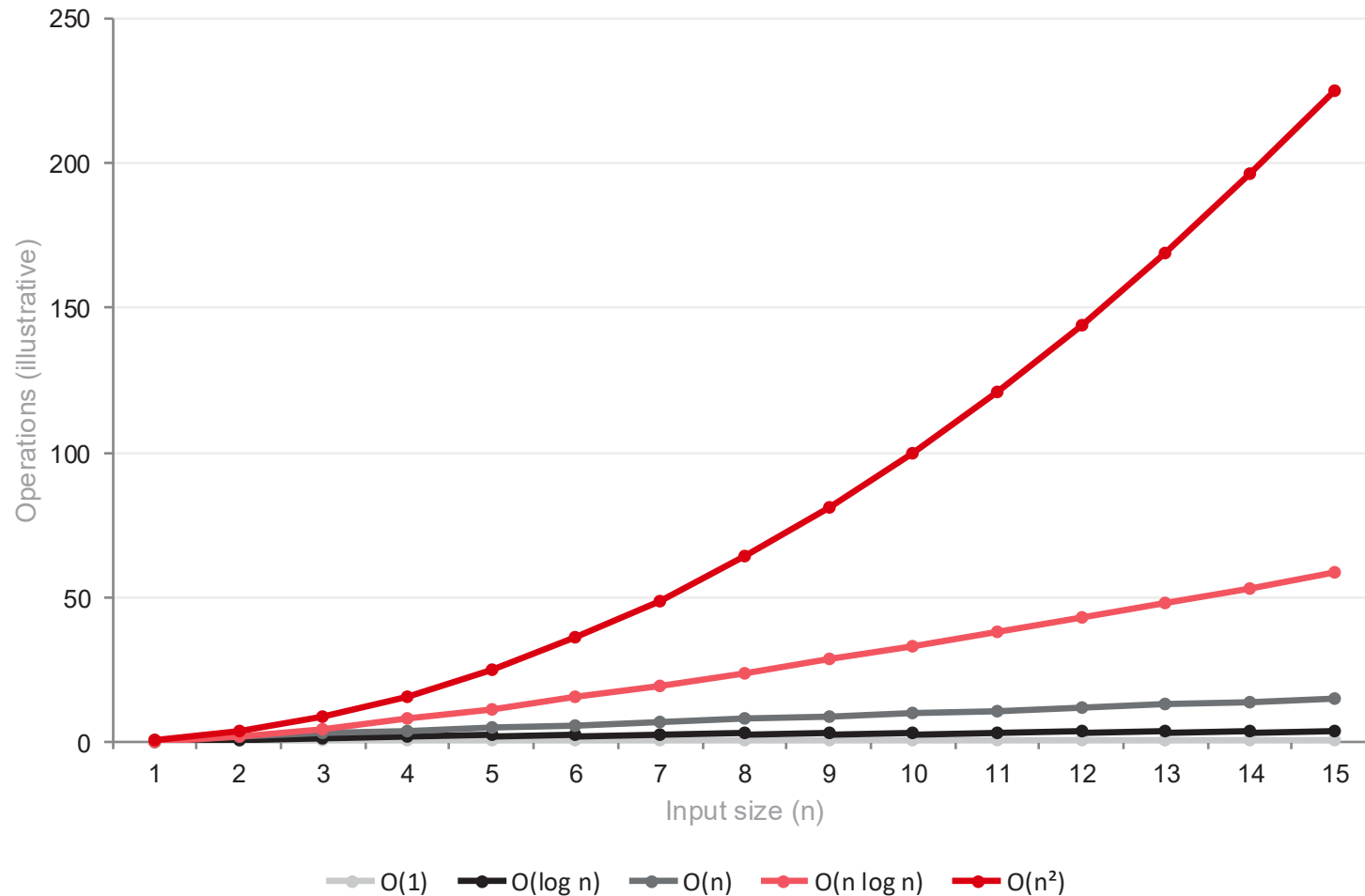
$O(n^2)$

Quadratic

Comparing every person in a room to every other person.

BIG-O NOTATION

How Growth Rates Compare



Why It Matters

As input size (n) grows, $O(n^2)$ algorithms slow down dramatically - while $O(1)$ and $O(\log n)$ barely notice.

This is exactly why choosing the right data structure - the topic of the rest of this deck - matters so much in practice.

Data structures

Pick the right tool - Big-O works for you,
not against you.



Five Building Blocks You'll Use Constantly



list

Ordered & changeable
- like a shopping list



dict

Key → value pairs - like
a phone book



str

Text - a sequence of
characters



set

Unique & unordered -
like a bag of distinct
marbles



tuple

Ordered & locked - like
GPS coordinates



list - An Ordered, Changeable Collection

```
fruits = ["apple", "banana", "cherry"]
```

BASIC OPERATIONS



Search

```
"cherry" in fruits
```

 $O(n)$

Checked one item at a time until it's found (or the list ends).



Add

```
fruits.append("kiwi")
```

 $O(1)^*$

Placed right after the last item. * $O(n)$ if inserted at the front or middle - the rest must shift.



Delete

```
fruits.remove("banana")
```

 $O(n)$

Python finds it first, then shifts every later item one step left.



dict - Key → Value Pairs (like a phone book)

```
ages = {"Anna": 25, "Tom": 30}
```

BASIC OPERATIONS



Search

```
ages["Tom"]
```

O(1) avg

Hashing the key jumps straight to its value - no scanning needed.



Add

```
ages["Eva"] = 22
```

O(1) avg

The new key is hashed and placed directly into its slot.



Delete

```
del ages["Tom"]
```

O(1) avg

Hashing locates the key directly, so it's removed in one step.



str - An Immutable Sequence of Characters

```
name = "Python"
```

BASIC OPERATIONS



Search

```
"th" in name
```

 $O(n)$

Scanned character by character to look for a match.



Add

```
name + "!"
```

 $O(n)$

Strings can't change in place - a brand-new string is built from scratch.



Delete

```
name.replace("P", "")
```

 $O(n)$

Again, nothing is changed in place - a new string is built without it.



set - Unique, Unordered Items (no duplicates)

```
colors = {"red", "green", "blue"}
```

BASIC OPERATIONS



Search

```
"red" in colors
```

 $O(1)$ avg

Hashing checks directly for membership - no scanning required.



Add

```
colors.add("yellow")
```

 $O(1)$ avg

The new item is hashed and placed directly into its slot.



Delete

```
colors.remove("green")
```

 $O(1)$ avg

Hashing locates the item directly, so it's removed in one step.



tuple - An Ordered, Unchangeable Collection

```
point = (10, 20)
```

BASIC OPERATIONS



Search

```
10 in point
```

O(n)

Checked one item at a time, just like a list.



Add

```
point + (30,)
```

O(n)

Not allowed in place - tuples are frozen. This builds a whole new, bigger tuple.



Delete

```
point[:1]
```

O(n)

Same story - a new tuple is built without the removed item.

SUMMARY

The Cheat-Sheet: All Five, Side by Side

Structure	Search	Add	Delete	Ordered?	Duplicates?	Mutable?
list	$O(n)$	$O(1)^*$	$O(n)$	Yes	Yes	Yes
dict	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	Yes	Keys: No	Yes
str	$O(n)$	$O(n)^{**}$	$O(n)^{**}$	Yes	Yes	No
set	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	No	No	Yes
tuple	$O(n)$	$O(n)^{**}$	$O(n)^{**}$	Yes	Yes	No

* $O(1)$ when adding to the end of a list; $O(n)$ when inserting at the front or middle.

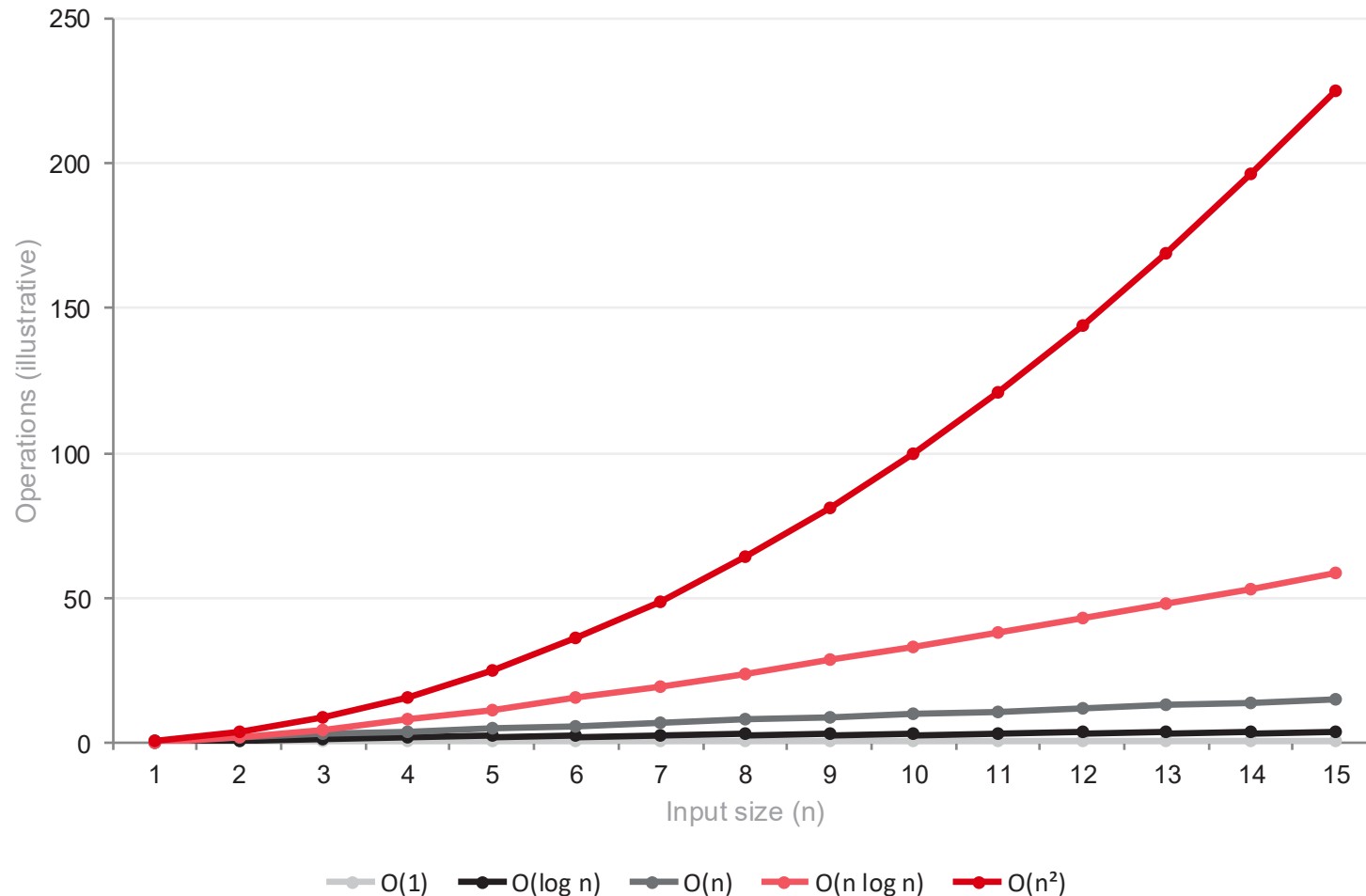
** Creates a brand-new object - the original string or tuple is never changed (that's what "immutable" means).

Time & memory

So how do you actually figure out the Big-O of a piece of code?



How do you actually calculate Big-O?



Why It Matters

As input size (n) grows, $O(n^2)$ algorithms slow down dramatically - while $O(1)$ and $O(\log n)$ barely notice.

This is exactly why choosing the right data structure - the topic of the rest of this deck - matters so much in practice.

How do you actually calculate Big-O?

1

Rule 1: Count the loops

No loop $\rightarrow O(1)$

One loop over $n \rightarrow O(n)$

Nested loop over $n \rightarrow O(n^2)$

2

Rule 2: Drop the constants

$O(2n) \rightarrow O(n)$

$O(n + 100) \rightarrow O(n)$

$O(3n^2) \rightarrow O(n^2)$

3

Rule 3: Keep only the biggest term

$O(n^2 + n) \rightarrow O(n^2)$

$O(n + \log n) \rightarrow O(n)$

"Winner " stays. The rest disappears.

How do you actually calculate Big-O?

1

Rule 1: Count the loops

```
def has_pair_sum(lst, target):  
    for i in range(len(lst)):  
        for j in range(i+1, len(lst)):  
            if lst[i] + lst[j] == target:  
                return True  
    return False
```

Loop inside a loop $\rightarrow O(n \cdot n) = O(n^2)$

2

Rule 2: Drop the constants

```
def process(lst):  
    for x in lst: # O(n)  
        print(x)  
    for y in lst: # O(n)  
        print(y)
```

Separate $\rightarrow O(n + n) = O(2n) \rightarrow O(n)$

Not nested. Constants disappear.

3

Rule 3: Keep only the biggest term

```
def analyze(lst):  
    for x in lst: # O(n)  
        print(x)  
    for i in range(len(lst)): # O(n^2)  
        for j in range(len(lst)):  
            print(lst[i], lst[j])
```

Total: $O(n) + O(n^2) = O(n^2 + n) \rightarrow O(n^2)$

The bigger term "eats" the smaller one.

6-step procedure for any problem

1

Understand

Read the problem twice, restate it in your own words - out loud if you can.

2

Example

Walk through a small example by hand before writing any code.

3

Edge cases

Ask: empty input? One element? Duplicates? Negative numbers?

4

Brute force

Always start here. A slow, honest solution beats a fast, broken one.

5

Optimize

Ask "can a data structure help?" - that single question unlocks most optimizations.

6

Test

Dry run your code in your head on the example from step 2 before calling it done.



Live coding workshops

One problem, one pattern, a whole family of solutions unlocked.

Answer “yes” or “no” to yourself:

Have you heard
of **Big-O**/ time
complexity
before?

- 3 × YES -  **Advanced**

Have you
ever solved
a **LeetCode**
problem?

- 0–2 × YES -  **Beginners**

Do you
know when to
use a dict/set
instead of
a list?

- Unsure? -  **Beginners**
(you can always jump tracks)

Your turn!

This is where the pattern becomes yours,
not just something you watched.

1480.

Running Sum of 1d Array

1672.

Richest Customer Wealth

412.

Fizz Buzz



Advanced path - Best Time to Buy and Sell Stock

121.

122.

123.

188.

3573.



KEY TAKEAWAYS

Five Things to Remember

- ✓ Big-O describes how performance scales - not exact seconds.
- ✓ dict & set use hashing → about $O(1)$ average for search, add, and delete.
- ✓ list & str are ordered but searching them costs $O(n)$.
- ✓ tuple is list's unchangeable twin - use it for data that should never change.
- ✓ There's no single “best” structure - the right choice depends on what you do most often.

Thank You

Thank you!

